AWS Cloud Practitioner Essentials

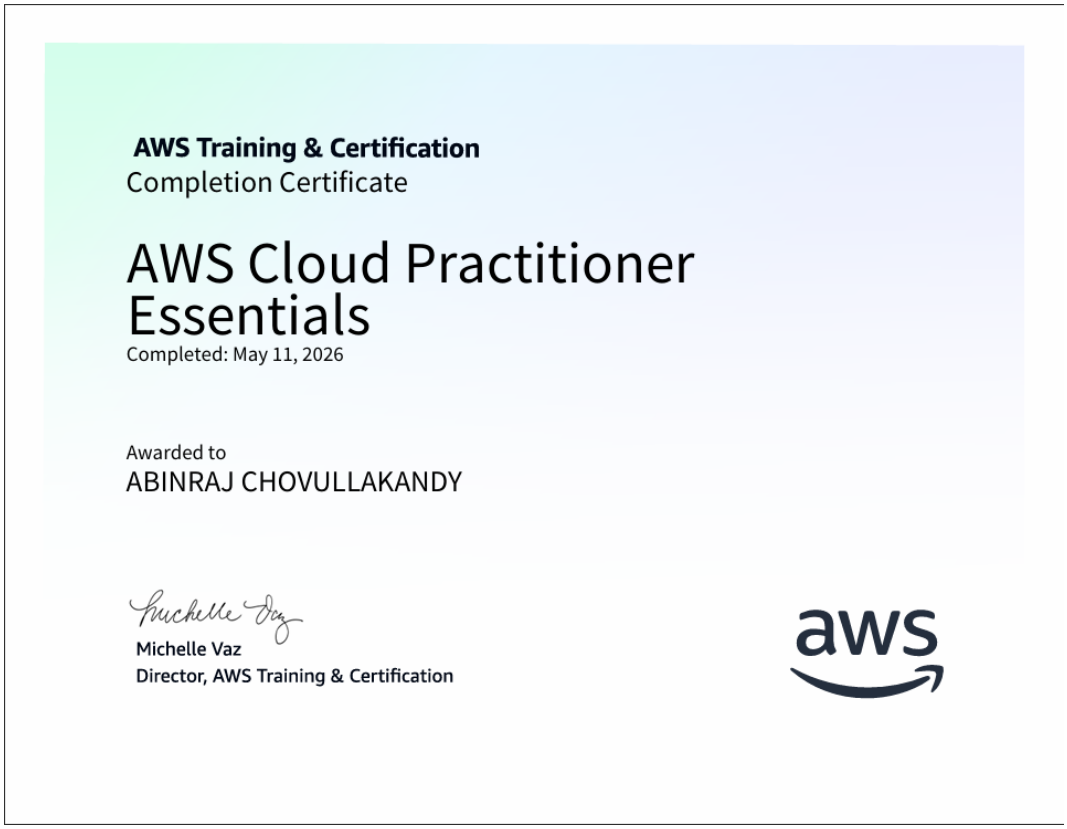
Course Assignment — IoT and Cloud Computing

Student	ABINRAJ CHOVULLAKANDY
Course	IoT and Cloud Computing
University	Riga Nordic University
Completed	MAY 11,2026
Scenario	Scenario 2 — Telecom CDR Analytics Platform

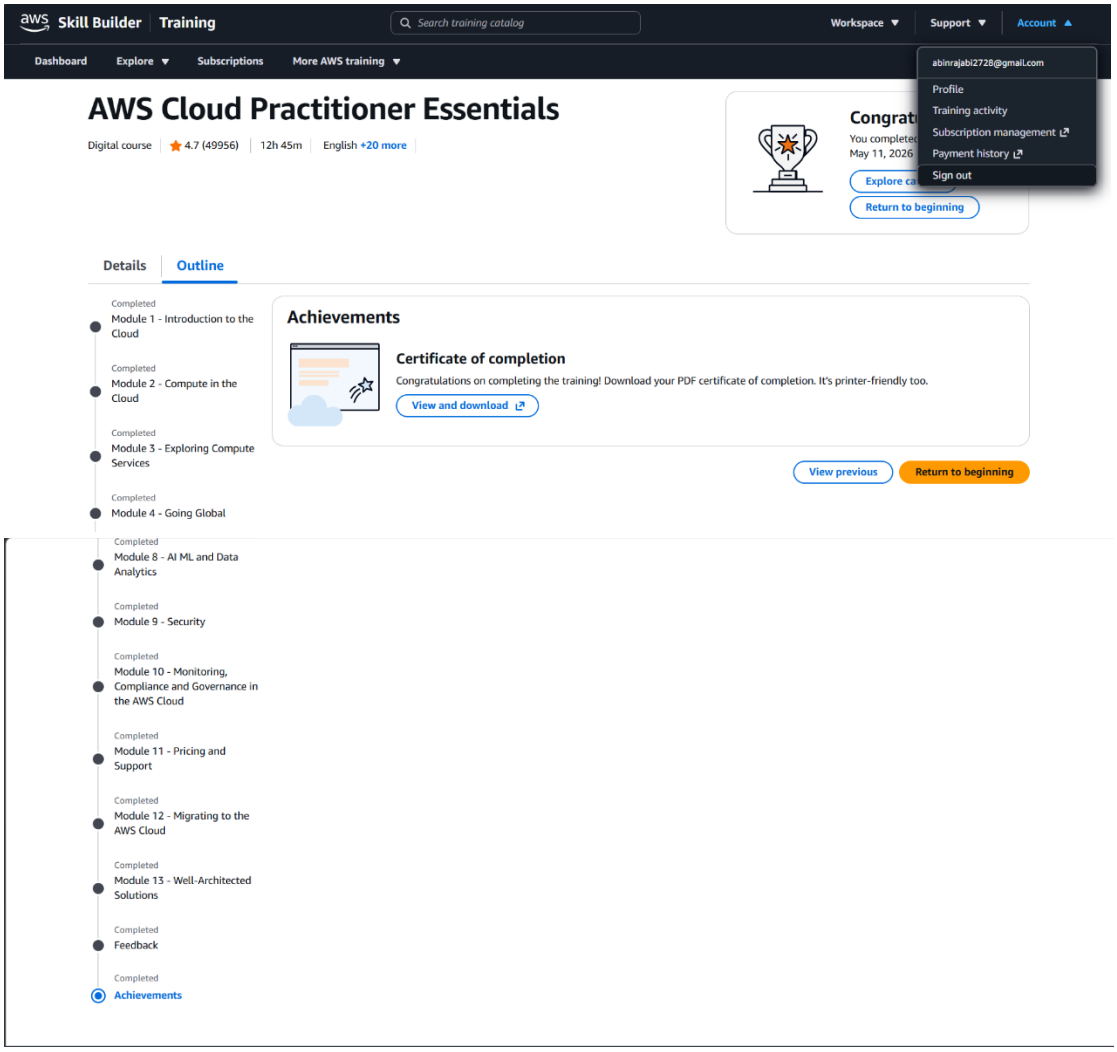
PART A — Completion Evidence

The following evidence confirms successful completion of the AWS Cloud Practitioner Essentials course on AWS Skill Builder:

- Certificate of completion awarded to ABINRAJ CHOVULLAKANDY— dated MAY 11,2026 (PDF certificate attached).



- AWS Skill Builder dashboard screenshot showing all 13 modules marked as Completed, including Modules 7–13 and the Achievements section visible in the submission screenshot.



- The student name is clearly visible on the completion certificate as issued by AWS Training & Certification, signed by Michelle Vaz, Director of AWS Training & Certification.

PART B — Module Reflection Journal

The following section provides a structured reflection for each of the 13 modules completed in the AWS Cloud Practitioner Essentials course. Each reflection is organised into four sections: Key Concepts, Technical Deep Dive, IoT/Telecom Connection, and Critical Reflection.

Module 1 — Introduction to Amazon Web Services

Key Concepts

#	Key Concept
1	Distinction between on-premises infrastructure and cloud-delivered services — shifting from capital to operational expenditure.
2	Three primary cloud deployment models: public cloud, private cloud, and hybrid cloud.
3	The AWS global value proposition: no upfront hardware investment, pay only for what you consume.
4	Overview of the AWS global infrastructure — Regions, Availability Zones, and Edge Locations.
5	The client-server model as the foundational interaction pattern for cloud services.

Technical Deep Dive — Amazon EC2

Amazon Elastic Compute Cloud (Amazon EC2) provides resizable virtual machines — called instances — that run on physical servers housed in AWS data centres. When a user launches an instance, AWS selects a physical host from its fleet, creates a virtualised environment using the Nitro System hypervisor, and maps that virtual machine to the user's request. The chosen instance type (e.g., t3.micro, m6i.large) determines vCPU count, RAM, and network bandwidth. Instances run Amazon Machine Images (AMIs) — pre-configured templates containing the operating system and software. EC2 pricing offers four main models: On-Demand (pay per second/hour, no commitment), Reserved Instances (1- or 3-year commitments with up to 72% savings), Spot Instances (spare capacity at up to 90% discount, subject to interruption), and Savings Plans (flexible commitment-based discounts). For experimentation, On-Demand is appropriate; for stable production workloads, Reserved Instances yield significant savings.

IoT / Telecom Connection

In an IoT monitoring context, EC2 instances serve as the processing layer for incoming sensor data. Imagine a network of air quality sensors across a city — each sensor sends readings via MQTT to an AWS IoT Core endpoint. EC2 instances running a data processing application can subscribe to those topics, apply threshold logic, and trigger alerts when pollution levels exceed safe limits. In a telecom context, EC2 instances could host Virtual Network Functions (VNFs) that process signalling traffic or mediate between legacy and cloud-native network elements.

Critical Reflection

What surprised me most was how AWS frames cloud computing around economic transformation rather than purely technical benefit. The shift from capital to operational expenditure has profound implications for how businesses plan and scale. What remains less clear is how the Nitro

hypervisor compares to traditional hypervisors like KVM in measurable performance benchmarks. I would explore EC2 bare-metal instances further, as they appear to offer near-native performance while still being fully managed through the AWS API.

Module 2 — Compute in the Cloud

Key Concepts

#	Key Concept
1	EC2 instance families and their specialised purposes: compute-optimised, memory-optimised, storage-optimised, and accelerated computing.
2	EC2 Auto Scaling, which dynamically adjusts the number of running instances in response to demand changes.
3	Elastic Load Balancing (ELB), which distributes incoming traffic across multiple healthy instances.
4	AWS Lambda — serverless compute that executes code without provisioning or managing servers.
5	Amazon ECS and Amazon EKS for running containerised workloads in a managed environment.

Technical Deep Dive — AWS Lambda

AWS Lambda is a serverless compute service that executes code in response to events without requiring the user to manage any underlying infrastructure. Lambda functions are packaged as deployment packages (ZIP or container images) and associated with an IAM execution role. When an event source — such as an API Gateway request, an Amazon S3 object upload, or a message in an Amazon SQS queue — triggers the function, Lambda initialises a micro-VM using Firecracker (a lightweight virtualisation technology), loads the function code and runtime, and executes the handler. Lambda supports multiple runtimes including Python, Node.js, Java, and Go. Pricing is based on the number of invocations and duration in GB-seconds (memory allocated multiplied by execution time). The first 1 million requests per month and 400,000 GB-seconds are included in the AWS Free Tier, making Lambda extremely cost-effective for event-driven workloads.

IoT / Telecom Connection

Lambda is exceptionally well-suited to IoT architectures because sensor data arrives in bursts rather than at a constant rate. In a smart building monitoring system, hundreds of occupancy sensors might simultaneously send readings at the top of every hour. Rather than maintaining always-on EC2 instances that sit idle between readings, Lambda functions can be triggered per sensor message, perform validation and enrichment, and write results to Amazon DynamoDB — scaling automatically from zero to thousands of concurrent executions. In telecom, Lambda could handle real-time preprocessing of network alarm events, transforming raw SNMP traps into structured, actionable records.

Critical Reflection

The cold start latency of Lambda — the delay when a function has not been invoked recently and AWS must initialise a new execution environment — was something I had not fully appreciated

before this module. For latency-sensitive applications this could be problematic. Provisioned Concurrency, which keeps a specified number of execution environments warm, addresses this at additional cost. I would explore how to architect Lambda-based IoT pipelines to minimise cold starts while keeping costs predictable, particularly for systems where response time is a hard requirement.

Module 3 — Global Infrastructure and Reliability

Key Concepts

#	Key Concept
1	AWS Regions — geographically isolated clusters of data centres designed to operate independently.
2	Availability Zones (AZs) — discrete, physically separated facilities within a Region, connected by high-bandwidth low-latency links.
3	The principle that deploying across multiple AZs provides resilience against single-facility failures.
4	AWS Edge Locations and Amazon CloudFront, which cache content closer to end users globally to reduce latency.
5	AWS Local Zones and Wavelength Zones, which bring compute to the network edge for ultra-low-latency use cases such as 5G applications.

Technical Deep Dive — Amazon CloudFront

Amazon CloudFront is AWS's Content Delivery Network (CDN), distributing content across over 450 Points of Presence globally. When a user requests content, CloudFront routes the request to the nearest edge location using AWS Anycast routing. If the content is already cached at that edge location, it is served directly without reaching the origin. If not, CloudFront fetches from the origin (an Amazon S3 bucket, an Application Load Balancer, or a custom HTTP origin), caches it per configured TTL rules, and serves it to the user. CloudFront integrates with AWS Shield for DDoS protection and AWS WAF for application-layer security filtering. Pricing is based on the volume of data transferred out and the number of HTTP requests processed, varying by geographic region served.

IoT / Telecom Connection

In a telecom context, CloudFront can serve firmware update packages to large fleets of IoT devices distributed across multiple countries. Rather than routing all devices to a single origin server — creating bandwidth bottlenecks and high latency for geographically distant devices — CloudFront caches the firmware binary at edge locations. A device in Singapore and a device in Latvia both receive the update quickly from a nearby edge location. This also substantially reduces egress costs from the origin, since only the first cache miss per edge location incurs full origin data transfer charges.

Critical Reflection

What surprised me was the sheer number of AWS edge locations compared to Regions — over 450 Points of Presence versus fewer than 35 geographic Regions. The global reach of CloudFront

is genuinely impressive. What remains somewhat unclear is the exact decision logic AWS uses to select a Regional Edge Cache versus a standard Edge Location when content is not locally cached. I would investigate CloudFront cache behaviours and Origin Shield configurations more deeply, particularly for architectures serving high-throughput IoT telemetry dashboards to globally distributed users.

Module 4 — Networking

Key Concepts

#	Key Concept
1	Amazon Virtual Private Cloud (Amazon VPC) — a logically isolated network environment in AWS where resources are launched.
2	Subnets — public (reachable from the internet via an Internet Gateway) and private (internal only, accessed via NAT Gateway for outbound traffic).
3	Internet Gateways and NAT Gateways controlling inbound and outbound internet connectivity respectively.
4	Security Groups (stateful, instance-level) and Network ACLs (stateless, subnet-level) as two layers of network access control.
5	AWS Direct Connect, which provides a dedicated private network connection from on-premises infrastructure to AWS.

Technical Deep Dive — Amazon VPC

Amazon VPC allows users to define a logically isolated section of the AWS cloud with a private IP address range (CIDR block, e.g., 10.0.0.0/16). The VPC is subdivided into subnets, each associated with a specific Availability Zone. Public subnets have a route to an Internet Gateway; private subnets route outbound traffic through a NAT Gateway placed in a public subnet. Security Groups act as stateful virtual firewalls at the instance level — inbound rules automatically permit corresponding outbound responses. Network ACLs operate at the subnet boundary and are stateless, requiring explicit rules for both inbound and outbound traffic. VPC itself is free; charges apply for components such as NAT Gateways (hourly rate plus per-GB data processing fee) and VPN connections.

IoT / Telecom Connection

In an IoT platform, VPC design is critical for security and scalability. A well-designed VPC places AWS IoT Core endpoints in public-facing tiers, while backend processing services (Lambda, EC2) and databases (Amazon RDS, DynamoDB) reside in private subnets. For a telecom operator migrating network management systems to AWS, Direct Connect provides the dedicated, low-latency link necessary for managing production network elements — internet-based VPN would introduce unacceptable latency and reliability concerns for real-time network operations.

Critical Reflection

The distinction between Security Groups (stateful) and Network ACLs (stateless) was initially confusing. In practice, most architectures rely primarily on Security Groups for fine-grained access control and use Network ACLs only as a broad perimeter filter. What I found interesting is that a

single Security Group can be associated with multiple instances, enabling centralised policy management. I would explore VPC Peering and AWS Transit Gateway further, as they address connectivity between multiple VPCs — a common requirement in large enterprise and telecom environments with dozens of AWS accounts.

Module 5 — Storage and Databases

Key Concepts

#	Key Concept
1	Amazon S3 — object storage with virtually unlimited capacity and eleven nines (99.999999999%) of durability.
2	Amazon Elastic Block Store (Amazon EBS) — persistent block storage volumes attached to EC2 instances.
3	Amazon Elastic File System (Amazon EFS) — managed NFS file storage accessible simultaneously by multiple instances.
4	Amazon RDS — managed relational database service supporting MySQL, PostgreSQL, MariaDB, Oracle, and SQL Server.
5	Amazon DynamoDB — fully managed NoSQL key-value and document database delivering single-digit millisecond performance at any scale.

Technical Deep Dive — Amazon DynamoDB

Amazon DynamoDB is a serverless, fully managed NoSQL database service built on a distributed hash table architecture. Data is partitioned across multiple storage nodes using a partition key, ensuring reads and writes are distributed and no single node becomes a bottleneck. Each item is identified by a primary key (partition key alone, or a composite of partition key and sort key). DynamoDB replicates data synchronously across three AZs within a Region by default. For globally distributed applications, DynamoDB Global Tables extend replication across multiple Regions with active-active writes. The service supports two capacity modes: Provisioned (user specifies read/write capacity units) and On-Demand (AWS auto-scales, charges per request). DynamoDB Streams capture a time-ordered log of item-level changes, enabling event-driven architectures where Lambda functions process changes in near real-time.

IoT / Telecom Connection

DynamoDB is an excellent fit for storing IoT device state and telemetry metadata. Consider a telecom network with 100,000 managed devices — each device has a current state (online, offline, degraded) that updates frequently. DynamoDB can store the latest state per device using the device ID as partition key, and historical state transitions as items with a timestamp sort key. Its single-digit millisecond read latency means a network operations dashboard can query device status in real time without noticeable delay. DynamoDB Streams could trigger Lambda functions that send automated alerts when a device transitions to a degraded state.

Critical Reflection

The concept of hot partitions — where excessive traffic concentrates on a single partition because many items share the same partition key — was a subtle but important consideration. Choosing a

high-cardinality partition key (such as a hashed UUID) distributes load evenly. What I would explore further is DynamoDB PartiQL support, which allows SQL-compatible queries on DynamoDB tables — a useful bridge for teams transitioning from relational databases who find the native DynamoDB API unfamiliar.

Module 6 — Security

Key Concepts

#	Key Concept
1	The AWS Shared Responsibility Model — AWS secures the cloud infrastructure; customers are responsible for security within the cloud.
2	AWS Identity and Access Management (IAM) — managing identities and permissions across all AWS services.
3	IAM policies — JSON documents defining allowed or denied actions on specific resources using Effect, Action, Resource, and Condition.
4	Multi-Factor Authentication (MFA) as an additional layer of identity assurance beyond username and password.
5	The principle of least privilege — every user and service should have only the minimum permissions required to perform their function.

Technical Deep Dive — AWS IAM

AWS IAM is the centralised access control system for AWS. IAM users represent individuals or services; IAM groups collect users sharing the same permissions; IAM roles are assumed by AWS services, applications, or federated identities. Permissions are defined through IAM policies with a default-deny model: unless explicitly allowed, all actions are denied. When multiple policies apply, explicit Deny always overrides Allow. IAM roles are particularly important for service-to-service authentication — rather than embedding access keys in application code, an EC2 instance or Lambda function assumes a role, receiving temporary credentials that are automatically rotated by AWS. IAM is provided at no additional cost as part of the AWS account.

IoT / Telecom Connection

In an IoT platform, IAM roles are critical for securing the data pipeline. Each Lambda function that processes sensor data should assume a role with exactly the permissions it needs — for example, read from a specific SQS queue and write to a specific DynamoDB table — and nothing else. For a telecom CDR analytics platform, IAM resource-based policies on Amazon S3 buckets should restrict access to CDR files to only authorised analytics roles, with conditions requiring requests to originate from within the VPC. This prevents accidental or malicious exfiltration of sensitive subscriber call records.

Critical Reflection

What surprised me was the complexity that IAM policy evaluation can reach in large organisations — with service control policies, permission boundaries, and resource-based policies all interacting. The IAM Policy Simulator tool becomes essential in such environments. What I would explore further is AWS IAM Identity Center (formerly AWS SSO), which centralises access management

across multiple AWS accounts — a common need in enterprise and telecom deployments where dozens of AWS accounts may be in use simultaneously.

Module 7 — Databases

Key Concepts

#	Key Concept
1	The distinction between relational (SQL) and non-relational (NoSQL) databases and the workloads each suits best.
2	Amazon Aurora — a MySQL/PostgreSQL-compatible relational database built for the cloud with up to 5x the throughput of standard MySQL.
3	Amazon Redshift — a fully managed, petabyte-scale columnar data warehouse designed for analytical SQL queries.
4	Amazon DocumentDB — a MongoDB-compatible managed document database for flexible JSON data models.
5	Amazon Neptune — a managed graph database optimised for highly connected datasets such as social networks or knowledge graphs.

Technical Deep Dive — Amazon Redshift

Amazon Redshift is a columnar data warehouse optimised for analytical queries. Unlike row-oriented databases, Redshift stores each column separately, allowing queries that aggregate a subset of columns to read only relevant data from disk. Clusters consist of a leader node (which receives queries, generates execution plans, and coordinates parallel processing) and one or more compute nodes (which execute query fragments). Data is distributed across compute nodes using a distribution style (KEY, EVEN, or ALL) to minimise data movement during joins. Redshift integrates with Amazon S3 through the COPY command for bulk data loading. Amazon Redshift Serverless removes cluster-sizing complexity. Pricing is based on node type and count in provisioned mode, or RPU-hours in serverless mode.

IoT / Telecom Connection

Amazon Redshift is highly relevant for telecom CDR analytics. Call Detail Records are inherently analytical data — operators need to aggregate millions of records to compute metrics such as total call minutes per subscriber, revenue per region, or peak traffic hours. Loading CDRs daily from Amazon S3 into Redshift using the COPY command enables complex SQL analytics without impacting operational systems. For an IoT platform, Redshift could aggregate time-series readings from thousands of sensors into hourly or daily summaries for reporting and large-scale anomaly detection.

Critical Reflection

The concept of distribution keys and sort keys in Redshift — and how poorly chosen distribution can lead to data skew and performance degradation — was more nuanced than expected. It requires upfront schema design thinking that differs significantly from traditional relational database design. What I would explore further is Redshift Spectrum, which allows querying data directly in

Amazon S3 without loading it into Redshift clusters — enabling a data lake query pattern that could be very useful for archiving older CDR data cost-effectively.

Module 8 — AI, ML and Data Analytics

Key Concepts

#	Key Concept
1	Amazon SageMaker — an end-to-end platform for building, training, and deploying machine learning models at scale.
2	Amazon Rekognition — a computer vision service for image and video analysis without ML expertise required.
3	Amazon Comprehend — a natural language processing (NLP) service for sentiment analysis, entity recognition, and topic modelling.
4	Amazon Forecast — an ML-based time-series forecasting service for business metrics and IoT sensor data.
5	Amazon Kinesis — a family of services for real-time data streaming, processing, and analytics at any scale.

Technical Deep Dive — Amazon Kinesis Data Streams

Amazon Kinesis Data Streams is a scalable, durable real-time streaming service. A stream is composed of one or more shards, each capable of ingesting up to 1 MB per second and 1,000 records per second, and delivering up to 2 MB per second to consumers. Producers push records to the stream using the PutRecord or PutRecords API. Consumers — Lambda functions, Kinesis Data Analytics applications, or Kinesis Data Firehose delivery streams — read records in order within each shard using sequence numbers. Records are retained in the stream for 24 hours by default (up to 365 days). Kinesis Data Firehose extends this by automatically delivering streaming data to Amazon S3, Amazon Redshift, or Amazon OpenSearch Service, with optional Lambda-based transformation. Pricing is based on shard-hours and data volume written.

IoT / Telecom Connection

Kinesis Data Streams is a natural fit for the real-time ingestion layer of an IoT telemetry platform. In a smart city sensor network, thousands of devices continuously push temperature, humidity, and air quality readings. Kinesis Data Streams ingests this data in real time, buffers it, and feeds it simultaneously to a Lambda-based anomaly detector and a Firehose delivery stream storing raw readings in Amazon S3. In telecom, Kinesis could ingest real-time network performance metrics from base stations, feeding an Amazon Managed Service for Apache Flink application that detects degraded cell performance and triggers automated remediation workflows.

Critical Reflection

The shard model of Kinesis requires deliberate capacity planning — underprovisioning leads to throttling while overprovisioning wastes cost. Amazon Kinesis Data Streams On-Demand mode automatically scales shards, addressing this at a slightly higher per-record cost. What I found genuinely impressive was how Amazon Managed Service for Apache Flink allows complex

windowed aggregations on streaming data using SQL or Java/Python Flink applications — enabling real-time analytics without the batch ETL pipelines that traditional architectures require.

Module 9 — Security (Advanced)

Key Concepts

#	Key Concept
1	AWS Key Management Service (AWS KMS) — creating, managing, and rotating encryption keys for data at rest and in transit.
2	AWS Shield — managed DDoS protection at Standard (automatic, free) and Advanced (enhanced, paid) tiers.
3	AWS WAF (Web Application Firewall) — filtering malicious HTTP/S traffic at the application layer using managed and custom rules.
4	Amazon Inspector — automated security assessment of EC2 instances and container images for known vulnerabilities.
5	Amazon GuardDuty — a threat detection service that continuously monitors AWS account activity and network traffic using ML and threat intelligence.

Technical Deep Dive — Amazon GuardDuty

Amazon GuardDuty is a managed threat detection service that analyses AWS CloudTrail event logs, Amazon VPC Flow Logs, and DNS query logs using machine learning models and integrated threat intelligence feeds. GuardDuty runs entirely outside the customer's infrastructure — it receives log data through AWS's internal systems without requiring agents or changes to existing workloads. When GuardDuty identifies suspicious activity (an EC2 instance communicating with a known command-and-control server, unusual API calls from a new geography, or credential exfiltration patterns), it generates a finding categorised by severity. Findings route to Amazon EventBridge to trigger automated responses — such as isolating a compromised instance by modifying its Security Group. Pricing is based on volume of CloudTrail events and VPC Flow Logs analysed per month, with a 30-day free trial.

IoT / Telecom Connection

For a telecom CDR analytics platform, GuardDuty provides an essential security monitoring layer. Call Detail Records contain sensitive subscriber data — call timestamps, durations, and endpoint identifiers — making unauthorised access a serious regulatory concern. GuardDuty can detect anomalous patterns such as an IAM role that normally reads from a specific S3 bucket suddenly exfiltrating large volumes of data at 3 AM, or an EC2 analytics instance attempting outbound connections to suspicious external IP addresses. These detections enable rapid response before a breach escalates, directly supporting GDPR Article 33 requirements to notify authorities of data breaches within 72 hours.

Critical Reflection

What I found most insightful was the passive, agentless nature of GuardDuty — it requires no installation, no changes to running workloads, and activates within minutes of being enabled. This contrasts significantly with on-premises security monitoring tools that require substantial installation

and tuning. What remains unclear is how GuardDuty handles false positives in environments with legitimate but unusual access patterns, such as a large-scale data migration. I would explore trusted IP list configurations and the ability to suppress specific finding types in known-safe operational scenarios.

Module 10 — Monitoring, Compliance and Governance

Key Concepts

#	Key Concept
1	Amazon CloudWatch — monitoring and observability for collecting metrics, logs, and events from AWS resources and custom applications.
2	AWS CloudTrail — records all API calls made in an AWS account, providing a complete and tamper-evident audit trail.
3	AWS Config — continuously records resource configurations and evaluates them against desired-state compliance rules.
4	AWS Trusted Advisor — an automated advisor checking cost optimisation, security, fault tolerance, performance, and service quotas.
5	AWS Organizations — centrally managing multiple AWS accounts with consolidated billing and service control policies.

Technical Deep Dive — Amazon CloudWatch

Amazon CloudWatch is the observability backbone of AWS. It collects metrics — numerical time-series data points such as CPU utilisation, network bytes in/out, or DynamoDB consumed read capacity — from virtually all AWS services automatically. Users can also publish custom metrics from their own applications using the PutMetricData API. CloudWatch Logs ingests, stores, and enables querying of log data from Lambda functions, EC2 instances (via the CloudWatch Agent), and AWS services. CloudWatch Logs Insights provides an interactive query language for analysing log data at scale. CloudWatch Alarms trigger actions (SNS notifications, Auto Scaling adjustments, EC2 actions) when a metric crosses a defined threshold. Pricing is based on the number of custom metrics, volume of log data ingested and stored, and number of alarms configured.

IoT / Telecom Connection

In an IoT platform, CloudWatch becomes the centralised observability layer. Custom metrics can be published from the data ingestion pipeline — readings processed per minute, percentage of readings failing validation, or end-to-end latency from sensor transmission to database write. CloudWatch Alarms alert operations teams when latency exceeds acceptable thresholds. For a telecom CDR analytics platform, CloudTrail ensures that every access to CDR files in Amazon S3 is logged with the IAM identity, timestamp, and source IP — providing the audit trail necessary for regulatory compliance under GDPR and the European Electronic Communications Code.

Critical Reflection

The integration between CloudWatch, CloudTrail, and AWS Config creates a powerful governance triad: CloudWatch tells you what is happening now; CloudTrail tells you who did what and when; AWS Config tells you whether infrastructure matches its desired state. What I found less intuitive

was the difference between CloudWatch Logs metric filters (which extract metric values from log patterns as persistent time-series data) and CloudWatch Logs Insights queries (which are ad-hoc investigations). Both analyse logs but serve fundamentally different operational purposes.

Module 11 — Pricing and Support

Key Concepts

#	Key Concept
1	The AWS pay-as-you-go pricing model — no upfront commitments, charged per unit of consumption with no minimum spend.
2	Three recurring pricing dimensions across most AWS services: compute time, data stored, and data transferred out.
3	The AWS Free Tier — 12 months of limited free access to many services, plus always-free tiers for select services.
4	AWS Cost Explorer and AWS Budgets — tools for visualising, analysing, and controlling AWS spending proactively.
5	Four AWS Support Plans — Basic, Developer, Business, and Enterprise — with escalating response times and access to senior AWS engineers.

Technical Deep Dive — AWS Cost Explorer

AWS Cost Explorer is a visualisation and analysis tool for understanding AWS spending. It provides pre-built reports showing cost and usage broken down by service, linked account, Region, tag, or usage type. Users can apply filters and group-by dimensions to identify which services or teams are driving cost growth. Cost Explorer also provides Reserved Instance and Savings Plans utilisation reports, helping identify whether existing commitments are being fully used. The Rightsizing Recommendations feature analyses EC2 instance utilisation metrics from CloudWatch and suggests downsizing consistently underutilised instances. Cost Explorer's API allows programmatic retrieval of cost data for integration with internal financial reporting systems. Cost Explorer itself is free; a charge applies per API request made.

IoT / Telecom Connection

Cost management is particularly important in IoT and telecom environments because both involve high data volumes that can generate unexpectedly large AWS bills. An IoT platform ingesting readings from 100,000 sensors generates significant Kinesis, Lambda, DynamoDB, and S3 costs. AWS Budgets can be configured to alert the operations team when monthly spend is projected to exceed a defined threshold, preventing bill shock. For a telecom CDR platform processing 50 million records daily, Cost Explorer's usage type breakdown can reveal whether Redshift query costs, S3 data transfer charges, or Lambda invocation costs are the primary driver — enabling targeted optimisation efforts.

Critical Reflection

What I found surprising was how significantly data egress costs — charges for transferring data out of AWS to the internet or to other Regions — can accumulate in data-heavy architectures. Inbound data transfer is free, but outbound is not. This asymmetry has real design implications: keeping

processing close to data storage (for example, using Redshift Spectrum to query S3 data in the same Region rather than copying it out) can meaningfully reduce costs. I would examine AWS data transfer pricing carefully before finalising any IoT or CDR analytics architecture.

Module 12 — Migrating to the AWS Cloud

Key Concepts

#	Key Concept
1	The AWS Cloud Adoption Framework (AWS CAF) — organising migration guidance across six perspectives: Business, People, Governance, Platform, Security, and Operations.
2	The 7 Rs of migration strategies: Retire, Retain, Rehost, Replatform, Repurchase, Refactor, and Relocate.
3	AWS Migration Hub — a central location to track and manage migration progress across multiple AWS and partner tools.
4	AWS Database Migration Service (AWS DMS) — migrating databases to AWS with minimal downtime using continuous replication.
5	AWS Snow Family (Snowcone, Snowball Edge, Snowmobile) — physically transporting large data volumes to AWS when network transfer is impractical.

Technical Deep Dive — AWS Database Migration Service

AWS Database Migration Service (AWS DMS) enables migration of databases to AWS with minimal downtime. It supports both homogeneous migrations (e.g., Oracle to Oracle on Amazon RDS) and heterogeneous migrations (e.g., SQL Server to Amazon Aurora PostgreSQL). A replication instance — a managed EC2 instance provisioned by DMS — connects to both source and target databases. For full-load migrations, DMS reads all existing data from the source and writes it to the target. For ongoing replication (change data capture), DMS reads the source database transaction log (Oracle redo logs, MySQL binlog) and continuously applies changes to the target, maintaining synchronisation until cutover. DMS pricing is based on replication instance hours and the volume of log storage used during the migration.

IoT / Telecom Connection

For a telecom operator migrating an on-premises CDR batch analytics system to AWS, DMS could migrate an existing Oracle data warehouse to Amazon Redshift or Amazon Aurora, with the AWS Schema Conversion Tool (SCT) translating stored procedures and schema objects to target equivalents. This enables a phased migration where the on-premises system remains operational during the migration window, reducing risk. For IoT platform migrations with years of historical sensor data on local servers, the AWS Snow Family is relevant for the initial bulk data load — shipping a Snowball Edge device is far faster and cheaper than transferring petabytes over an internet connection.

Critical Reflection

The 7 Rs framework was more nuanced than I initially expected. The difference between Rehost (lift-and-shift with no changes) and Replatform (targeted optimisations such as moving from self-

managed MySQL to Amazon RDS) is subtle but has significant implications for migration effort and the cloud-native benefits realised. Many organisations start with Rehost for quick wins, then gradually Refactor workloads over time. I would explore AWS Application Discovery Service further — automated discovery of on-premises application dependencies seems essential before committing to a migration strategy.

Module 13 — Well-Architected Solutions

Key Concepts

#	Key Concept
1	Operational Excellence — running and monitoring systems to deliver business value, and continually improving processes and procedures.
2	Security — protecting data, systems, and assets using cloud security best practices including least privilege and encryption.
3	Reliability — ensuring workloads perform their intended functions correctly and recover quickly from failures.
4	Performance Efficiency — using computing resources efficiently and adapting as demand and technology evolve.
5	Cost Optimisation — avoiding unnecessary costs by understanding spending and selecting the right resource types.
6	Sustainability — minimising the environmental impact of running cloud workloads by maximising utilisation and adopting efficient technologies.

Technical Deep Dive — AWS Well-Architected Tool

The AWS Well-Architected Tool is a managed service that helps customers review architectures against the six pillars of the Well-Architected Framework. Users define a workload and answer a standardised questionnaire covering design decisions across all six pillars. For each question, the tool identifies risks (High Risk Issues and Medium Risk Issues) and provides improvement plan guidance with links to relevant AWS documentation. The tool generates a Workload Summary Report that can be shared with stakeholders. It supports multiple lenses — including the Serverless Lens, Container Build Lens, Machine Learning Lens, and IoT Lens — which extend the core framework with domain-specific best practices. The AWS Well-Architected Tool is available at no charge.

IoT / Telecom Connection

The IoT Lens of the Well-Architected Tool is directly applicable to IoT architectures. It addresses IoT-specific reliability concerns such as device connectivity interruptions (offline-first device design, message queuing with AWS IoT Core), security concerns for constrained devices (certificate-based authentication, over-the-air certificate rotation), and operational excellence considerations (fleet-level monitoring, remote device management). For a telecom CDR analytics platform, the Data Analytics Lens provides guidance on building secure, performant, and cost-efficient pipelines — directly applicable to the Scenario 2 analysis in Part C.

Critical Reflection

What struck me most about the Well-Architected Framework is its deliberate non-prescriptiveness — it defines principles and asks design questions without mandating specific AWS services. This reflects the reality that good cloud architecture is contextual. What remains somewhat unclear is how to prioritise trade-offs between pillars when they conflict — adding redundancy for reliability increases cost, which conflicts with cost optimisation. The framework acknowledges these trade-offs but leaves resolution to the architect's judgement, which requires real-world experience to develop confidently.

PART C — Scenario Analysis

Scenario 2: Telecom CDR Analytics Platform — Cloud-Native AWS Architecture

1. Introduction

A regional telecommunications operator currently processes 50 million Call Detail Records (CDRs) per day on on-premises batch analytics infrastructure. CDRs are structured records generated for every voice call, SMS, and data session on the network, capturing fields such as originating number, terminating number, call start and end timestamps, duration, bytes transferred, and serving cell identity. These records are foundational to revenue assurance, fraud detection, regulatory reporting, and customer experience analytics.

Migrating this workload to AWS presents an opportunity to improve scalability, reduce operational overhead, and enable near-real-time analytics capabilities that batch on-premises systems cannot support. This analysis proposes a cloud-native AWS architecture, evaluates appropriate storage and database services, examines the Shared Responsibility Model implications for sensitive CDR data, and compares pricing strategies for the expected workload.

2. Proposed Cloud-Native Architecture

The architecture follows a layered pipeline: ingestion, storage, processing, analytics, and visualisation. The table below summarises the principal components and their corresponding AWS services.

Component	AWS Service / Design Decision
Ingestion Transport	AWS Direct Connect — dedicated private link from operator data centre to AWS; CDR files never traverse the public internet.
Raw Landing Zone	Amazon S3 with S3 Object Lock (WORM mode) — tamper-proof storage satisfying regulatory record retention requirements.

Component	AWS Service / Design Decision
File Arrival Trigger	Amazon S3 Event Notifications — triggers an AWS Lambda function upon each new CDR file arrival for format validation and decoding.
Streaming Buffer	Amazon Kinesis Data Streams — decouples ingestion from processing; absorbs delivery bursts common in telecom batch delivery windows.
Data Enrichment	AWS Lambda transformation function (via Kinesis Data Firehose) — joins subscriber tier data from Amazon DynamoDB; flags anomalous records for fraud review; converts to Apache Parquet format.
Processed Data Lake	Amazon S3 (partitioned by year/month/day/hour using S3 prefix structure) — columnar Parquet format for efficient downstream query performance.
Analytics Warehouse	Amazon Redshift — columnar MPP warehouse; CDR fact tables distributed by hashed MSISDN; sort keys on call timestamp for efficient range-based filtering.
Historical Queries	Amazon Redshift Spectrum — queries CDR data older than 90 days directly in S3 without loading into Redshift, reducing cluster storage costs significantly.
Ad-hoc Exploration	Amazon Athena — serverless SQL queries on the S3 data lake using AWS Glue Data Catalog; pay per TB scanned; ideal for data science teams.
Visualisation	Amazon QuickSight — self-service BI dashboards for revenue assurance, regulatory, and network operations teams; SPICE caches frequently accessed datasets.
Subscriber Reference	Amazon DynamoDB — stores current subscriber tier and enrichment data; single-digit millisecond lookups during real-time CDR enrichment pipeline.
Security Monitoring	Amazon GuardDuty + AWS CloudTrail — threat detection and full API audit trail for GDPR Article 33 compliance and regulatory audits.

3. Storage and Database Service Justification

Amazon S3 is the foundational storage layer for both the raw CDR landing zone and the processed data lake. S3's eleven nines of object durability, effectively unlimited capacity, and fine-grained access control via bucket policies and IAM make it the appropriate choice for storing CDR data at scale. S3 Intelligent-Tiering automatically moves objects between storage tiers based on access frequency, reducing costs for CDR data that is accessed frequently in the first 30 days but rarely thereafter — a pattern highly characteristic of telecom analytics data.

Amazon DynamoDB serves the reference data and real-time subscriber lookup needs. The subscriber tier lookup during CDR enrichment requires millisecond latency; DynamoDB's on-demand scaling and consistent performance ensure this enrichment step does not become a bottleneck regardless of CDR ingestion rate or daily traffic peaks.

Amazon Redshift is the appropriate choice for the analytical data warehouse layer because CDR analytics workloads are inherently columnar and aggregative — computing total call minutes per

subscriber for a billing period, or identifying peak traffic hours across a region, requires scanning many rows but only a few columns. Redshift's columnar storage and massively parallel processing (MPP) architecture are purpose-built for exactly this access pattern.

4. Shared Responsibility Model for Sensitive CDR Data

The AWS Shared Responsibility Model delineates security obligations between AWS and the customer. AWS is responsible for security of the cloud — the physical security of data centres, server hardware, networking fabric, and managed service infrastructure. The telecom operator, as the customer, is responsible for security in the cloud — service configuration, access control, data encryption, and regulatory compliance.

For CDR data, this means the operator must implement the following controls:

- Server-side encryption on all S3 buckets using AWS KMS with customer-managed keys (CMKs), so the operator retains full control over the key lifecycle and can revoke access if required.
- Encryption at rest for all Amazon Redshift clusters and DynamoDB tables using AWS KMS CMKs.
- Encryption in transit using TLS for all API calls; AWS Direct Connect traffic encrypted using MACsec where the Direct Connect partner supports it.
- IAM least-privilege policies — the Lambda enrichment function's execution role has access only to the specific Kinesis stream, DynamoDB table, and S3 bucket it requires.
- Amazon Macie enabled on S3 CDR buckets — Macie uses ML to automatically discover and classify sensitive data (phone numbers, personal identifiers), alerting the security team if CDR data appears in unexpected S3 locations.
- Amazon GuardDuty for continuous threat detection; AWS CloudTrail for a complete, tamper-evident API audit trail supporting GDPR Article 33 breach notification obligations and national telecommunications regulatory authority audits.

5. Reserved vs. On-Demand Pricing Comparison

The expected workload for this platform is highly predictable: 50 million CDRs per day represents a stable baseline driven by the operator's subscriber base size, which changes slowly. This predictability makes Reserved Instance or Savings Plans commitments economically compelling for compute-intensive components. The table below summarises the recommended pricing model for each service layer.

Service	Pricing Model	Rationale
Amazon Redshift	Reserved Nodes (1-yr)	Predictable 24/7 workload — 32% saving vs On-Demand
Amazon Kinesis	On-Demand mode	CDR bursts are irregular; On-Demand avoids shard over-provisioning
AWS Lambda	On-Demand	Event-driven; scales to zero between CDR deliveries
Amazon DynamoDB	On-Demand	Reference lookups vary; avoids wasted provisioned capacity

Service	Pricing Model	Rationale
Amazon S3	Intelligent-Tiering	CDR data access drops sharply after 30 days; auto-tiering saves cost

For Amazon Redshift specifically, a Reserved Node commitment of one year offers approximately 32% savings versus On-Demand pricing for equivalent node types in eu-north-1 (Stockholm — appropriate for a Latvian operator). A 3-year commitment yields approximately 55% savings. Given that the Redshift cluster will run continuously as a production analytics platform, Reserved Node commitments represent a straightforward and significant cost optimisation. For Lambda, DynamoDB, and Kinesis, On-Demand pricing remains appropriate: these services scale to zero or near-zero during periods when CDRs are not being processed, and committing to provisioned capacity for variable workloads risks wasting spend on unused capacity.

6. Well-Architected Framework Alignment

This architecture directly addresses four of the six Well-Architected Framework pillars. Security: encryption at rest and in transit, IAM least privilege, Macie, GuardDuty, and CloudTrail satisfy the Security pillar requirements for protecting sensitive subscriber data. Reliability: using Direct Connect rather than internet VPN, deploying Kinesis with multiple shards, and storing raw CDRs in S3 with WORM mode ensures data is not lost even if downstream processing fails. Performance Efficiency: Redshift's columnar MPP architecture, Parquet format in S3, and Redshift Spectrum for cold data ensure queries remain performant as data volumes grow to petabyte scale. Cost Optimisation: Reserved Nodes for Redshift, S3 Intelligent-Tiering for the data lake, and serverless On-Demand pricing for variable-consumption services keeps total cost of ownership aligned with actual business value delivered.

7. Conclusion

The proposed architecture transforms a rigid on-premises batch analytics system into a scalable, secure, cloud-native CDR analytics platform. By using AWS Direct Connect for private ingestion, Amazon S3 with WORM mode for durable and compliant storage, Amazon Kinesis Data Streams for real-time buffering, Amazon Redshift for high-performance columnar analytics, and AWS KMS encryption throughout, the architecture addresses both the technical requirements and the regulatory obligations associated with sensitive subscriber data. The application of Reserved Node pricing for Redshift and On-Demand pricing for variable-consumption services optimises the cost structure for a workload that is predictable in its daily baseline but variable in its moment-to-moment delivery patterns.

References

- AWS Documentation — Amazon Web Services, Inc. https://docs.aws.amazon.com
- AWS Well-Architected Framework — https://aws.amazon.com/architecture/well-architected
- AWS Pricing Calculator — https://calculator.aws
- AWS Cloud Practitioner Essentials — AWS Skill Builder, completed MAY 11, 2026